GRAFIQ: Do VLMs Understand Image Formation?

Why this matters?

- VLMs can describe what is in an image, but not always why it looks that way
- This matters because VLMs are increasingly used as judges for image generation
- Strong semantic understanding does not guarantee understanding of graphics structure

What GRAFIQ does?

- Graphics-grounded benchmark testing whether VLMs understand rendering factors behind image formation
- Uses controlled synthetic scenes with known rendering parameters
- Generates QA automatically and verifies answers from render passes, not RGB

Benchmark categories

Category	What it tests	Example question
Geometry	Depth, shape, surface orientation	"Which object is closer to the camera?"
Materials	Glossiness, roughness, reflectance	"Which object has a shinier surface?"
Lighting	Shadows, shading, light direction	"Which object is more brightly illuminated?"
Viewing & Correspondence	Multi-view consistency	"Do these two images show the same scene?"

Two-Stage Pipeline with Two Scene Modes

Stage 1 – Controlled Rendering (Blender + Cycles):

- Use object pairs from Objaverse or generated primitives
- Vary one factor per scene; keep all others fixed
- Export RGB, task-specific render passes, and object masks
- Two rendering environments:

	Studio Mode	Complex Scene mode
Background	Flat gray background, shadow catcher	Infinigen indoor rooms
Object placement	Side-by-side floating objects	Objects placed on tables
Lighting	Controlled lamps	Scene lighting + added key light
Realism	Clean setting for isolated reasoning	Realistic setting with clutter and distractors

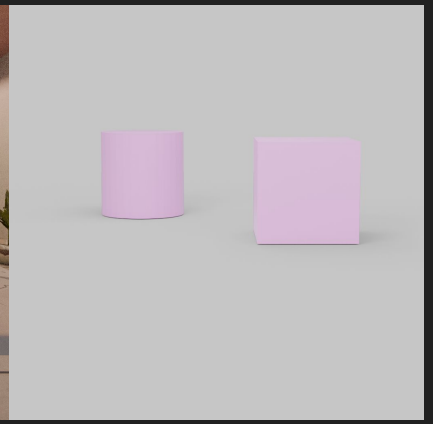
Stage 2 – QA Generation & Verification:

- Same QA pipeline for both scene modes
- Same question templates and verification logic
- Answer verified against ground-truth render passes; mismatches discarded

Results:

Complex Scene Depth Image

Studio Depth Image



Current Progress & Roadmap

✅ Completed

- Full rendering pipeline for 3 of 4 categories: Geometry (depth + normals), Materials (roughness), Lighting (light direction)
- Two scene modes implemented:
  a. Studio mode: 4 render scripts, simple flat-grey backgrounds
  b. Complex scene mode: 4 render_scene_*.py scripts using Infinigen procedural indoor environments
- Infinigen integration: submodule added, scene_utils extended with table detection, surface placement, Infinigen-compatible Cycles config
- Multi-task QA generator with pixel verification for all 4 task types (shared across both modes)
- All 4 complex-scene tasks producing verified QA items (depth, roughness, normal, lighting)
- Unit test suite validating all verifiers

⬜ Next Steps

- Viewing & Correspondence category (4th category — not yet implemented)
- Scale up: generate full benchmark dataset across diverse Infinigen scenes + Objaverse objects
- VLM evaluation: run GPT-5, Gemini, Claude, LLaVA, etc.
  a. report per-category accuracy
  b. studio-vs-complex-scene breakdown
- Error analysis: do VLMs struggle more in complex scenes than studio?
- (Stretch) Human baseline for comparison